# ChupIA: Assistente Inteligente para Acompanhamento do Pet

Documentação técnica atualizada — Sprint 4

## 1. Introdução

O ChupinVet é uma solução voltada ao acompanhamento e aos cuidados com animais de estimação, incluindo cadastro de pets, registros de rotina e agendamento de consultas. O ChupIA complementa essa proposta como assistente de Inteligência Artificial Generativa que permite consultar, em linguagem natural, informações cadastradas sobre um pet e seus registros de diário.

Nesta Sprint 4, o ChupIA possui um protótipo funcional em Python, executado no terminal, integrado à API Java real do ChupinVet. A interface final dentro do aplicativo permanece como possibilidade de integração futura.

## 2. Problema de Negócio

O acompanhamento contínuo de alimentação, consumo de água, humor e comportamento pode gerar vários registros por pet. Consultar e interpretar essas informações manualmente exige tempo e pode dificultar a identificação de alterações na rotina. O ChupIA busca facilitar o acesso e a compreensão dos dados já registrados, sem substituir a avaliação de um médico-veterinário.

## 3. Solução proposta

O responsável informa o ID do pet no terminal e faz perguntas em linguagem natural. Quando necessário, o modelo solicita a execução da função de consulta, que recupera dados cadastrais e registros do diário pela API Java. A resposta é construída com base nas informações efetivamente retornadas.

Exemplo validado: para o pet de ID 3, o ChupIA identificou a Yumi e, em uma pergunta seguinte, apresentou o registro de 05/06/2026, mantendo o contexto da conversa.

## 4. Abordagem de Inteligência Artificial

A implementação utiliza Python e a API do Google Gemini, com um modelo de linguagem (LLM) e Function Calling. A ferramenta declarada ao modelo chama-se consultar_dados_pet, recebe id_pet e consulta informações reais por meio do serviço Python. O retorno da ferramenta é enviado ao Gemini para elaboração da resposta.

O ChupIA utiliza instruções de sistema para responder em português, limitar-se aos dados fornecidos e evitar diagnósticos ou prescrições. A aplicação mantém o contexto entre mensagens por meio do identificador da interação anterior. Não foi implementado um mecanismo de RAG com base vetorial nesta versão.

## 5. Dados utilizados pelo ChupIA

Os dados são obtidos exclusivamente por endpoints da API Java, que realiza a comunicação com o banco Oracle. O serviço Python não acessa o Oracle diretamente e não realiza operações de escrita nos registros.

• POST /auth/login: autentica a conta de teste configurada no ambiente e retorna um token JWT.

• GET /pets/{id}: retorna informações cadastrais do pet, como nome, espécie, raça, idade e peso, quando disponíveis.

• GET /diarios/pet/{idPet}: retorna uma resposta paginada; os registros ficam no campo content e incluem dataRegistro, humor, alimentacao, consumoAgua, comportamento e pesoRegistrado.

O protótipo trabalha com os registros efetivamente disponibilizados pelos endpoints. Campos como sintomas e observações não foram presumidos como presentes no retorno real de diário testado.

## 6. Fluxo de Dados

• 1. O usuário executa python main.py e informa o ID do pet.

• 2. O usuário envia uma pergunta; o Gemini recebe a pergunta e o ID selecionado.

• 3. Quando necessário, o Gemini solicita a função consultar_dados_pet(id_pet).

• 4. O serviço Python autentica na API Java com as credenciais de teste configuradas e recebe um JWT.

• 5. Com o token, o serviço consulta os dados do pet e os registros do diário.

• 6. A API Java acessa o Oracle e devolve as informações disponíveis.

• 7. O serviço organiza os campos retornados e fornece o resultado da função ao Gemini.

• 8. O Gemini elabora a resposta e o terminal a apresenta; as próximas perguntas podem manter o contexto.

Fluxo implementado: Terminal → Serviço ChupIA (Python) → Gemini (chamada de função) → API Java (JWT) → Oracle → API Java → Serviço ChupIA → Gemini → Terminal.

## 7. Personalização

A resposta é contextualizada pelo ID informado ao iniciar a sessão e pelos registros associados ao pet consultado. A mesma pergunta pode produzir respostas diferentes conforme a existência e o conteúdo dos diários. O histórico da interação permite perguntas de acompanhamento, como “Quais são os registros do diário dele?”, após o nome do animal ter sido identificado.

## 8. Segurança e limitações do ChupIA

O assistente é informativo e não substitui atendimento veterinário. As instruções do modelo proíbem diagnósticos, prescrições, indicação de tratamentos e invenção de informações ausentes. Na falta de dados, o ChupIA deve informar essa ausência.

As chaves e credenciais são configuradas em arquivo .env, excluído do versionamento; o projeto inclui um .env.example sem segredos. A API Java exige autenticação JWT. Entretanto, os testes foram feitos com uma conta compartilhada de demonstração: a seleção de ID no terminal não equivale a uma validação própria de posse do pet. Para uso em produção, a autorização por usuário e a restrição do ID consultável precisam ser verificadas e aplicadas de ponta a ponta.

Em caso de indisponibilidade da API Java, o serviço retorna um erro genérico de consulta. A implementação atual não garante alta disponibilidade nem substitui monitoramento e tratamento de

falhas de produção.

## 9. Arquitetura de Solução

A aplicação está dividida em módulos: main.py (terminal e seleção do pet), config.py (configuração do Gemini), services/gemini_service.py (interações e Function Calling), tools/pet_tools.py (declaração da ferramenta), services/pet_service.py (organização do contexto), clients/java_api.py (requisições HTTP e autenticação) e prompts/system_prompt.py (regras do assistente).

| | |
|---|---|
| Usuário / Terminal | Pergunta + ID do pet |
| Gemini / Function Calling | Solicita consultar_dados_pet |
| Serviço Python | Login JWT e consultas HTTP |
| API Java | Acesso ao Oracle e retorno de PET/DIARIO |
| Serviço Python → Gemini | Contexto estruturado → resposta |
| Terminal | Exibe a resposta ao usuário |

Representação simplificada da arquitetura implementada. A integração visual com o aplicativo ChupinVet ainda não foi demonstrada neste protótipo.

## 10. Benefícios esperados

O ChupIA facilita a consulta de registros em linguagem natural, melhora a acessibilidade das informações do pet e ajuda o responsável a acompanhar dados de rotina. Para a equipe do projeto, demonstra a integração de IA generativa com serviços REST existentes, sem alterar a API Java ou consultar diretamente o banco Oracle.

## 11. Escopo inicial

O escopo entregue contempla: seleção de pet pelo ID, perguntas e respostas em terminal, integração com Gemini, Function Calling, autenticação JWT automática na API Java, leitura de dados cadastrais e diários, manutenção de contexto e orientações de segurança. Não contempla interface final integrada ao aplicativo, atualização de diários, prontuários clínicos ou consultas diretas ao Oracle.

## 12. Protótipo desenvolvido

A versão atual substituiu os dados simulados por chamadas reais à API Java hospedada em https://javachallenge.onrender.com. Os arquivos antigos do protótipo monolítico e do serviço de dados simulados foram removidos. A configuração é feita por variáveis de ambiente, incluindo GOOGLE_API_KEY, JAVA_API_URL, JAVA_API_EMAIL e JAVA_API_SENHA.

Para executar, instalar as dependências do requirements.txt, configurar o .env e rodar python main.py. O terminal solicita o ID do pet e recebe perguntas até o comando “sair”. A documentação do repositório (README) detalha o preparo do ambiente.

## 13. Resultados e validação

| ID | Pet | Resultado observado |
|---|---|---|
| 1 | Nasus | Dados consultados; 2 registros de diário. |
| 2 | Hecarim | Pet encontrado; 0 registros de diário. |
| 3 | Yumi | Pet encontrada; 1 registro de diário. |

O teste da Yumi confirmou continuidade de contexto entre duas perguntas e apresentação do diário de 05/06/2026: humor “Feliz”, alimentação “Normal”, consumo de água “Bebeu bem” e comportamento “Pet apresentou mais disposição para caminhadas”. Como o peso não estava informado, o assistente indicou sua ausência.

O teste do Hecarim confirmou que a resposta de ausência de registros correspondia à consulta real da API (totalElements = 0), e não a uma falha do Gemini. Esses cenários evidenciam consulta de dados existentes e tratamento de informações indisponíveis. Testes anteriores do protótipo também avaliaram a orientação para procurar um médico-veterinário em pedidos de diagnóstico; a integração final deve continuar sendo validada quanto a esse comportamento.

Conclusão: a Sprint 4 apresenta um protótipo funcional do ChupIA com dados reais, arquitetura modular e integração entre IA generativa e API REST. A evolução futura deverá priorizar a integração com a interface do ChupinVet e o reforço da autorização por usuário/pet.